Day 9: Caching, Eviction & Data Access Performance

Improving read performance while managing freshness and consistency.

Cache Fundamentals

- Stores frequently accessed data
- Reduces database load
- Requires a high hit rate

TTL, Refresh & Invalidation

- TTL expires old cache entries
- Refresh updates stale data
- Invalidation removes outdated data

Read & Write Strategies

- Cache-Aside
- Read-Through
- Write-Through

Eviction Policies (LRU, LFU, FIFO)

- LRU: Least recently used
- LFU: Least frequently used
- FIFO: First in, first out

Local vs Distributed Cache

- Local: Fast, limited to one node
- Distributed: Scalable, shared state
- Adds consistency and failure risks

Eviction Policies (LRU, LFU, FIFO)

- LRU: Least recently
- FIFU: Least frequently used

Local vs Distributed Cache

- Local: Fast, limited to one node
- Distributed: Scalable, shared state
- Adds consistency and failure risks

Hot Keys & Cache Stampedes

- Hot keys concentrate requests
- Cache stampedes overwhelm DBs
- Rate limiting & per key lock protection

Caching improves read performance by keeping hot data closer to the user.

Day 9: Caching, Eviction, and Data Access Performance

Day 9 focuses on how systems reduce repeated work and lower latency by storing hot data closer to the read path.

It covers:

- cache fundamentals
- cache-aside, read-through, and write-through patterns
- TTL, refresh, and invalidation
- LRU, LFU, and FIFO eviction policies
- cache consistency and stale reads
- local cache vs distributed cache
- hot keys and cache stampede protection

In short, Day 9 is about improving read performance without losing control of correctness, freshness, or operational cost.

Core ideas

Cache fundamentals

- A `cache` stores frequently accessed data so the system does not have to fetch or compute it every time.
- Caches reduce latency and lower load on databases or downstream services.
- A cache only helps when the hit rate is high enough to justify its complexity.

Read and write patterns

- `Cache-aside` lets the application load data into the cache on demand.
- `Read-through` hides cache misses behind the cache layer itself.
- `Write-through` updates the cache immediately with writes, while `write-back` delays persistence and is riskier.

TTL, invalidation, and freshness

- `TTL` limits how long cached data can stay before it expires.
- `Invalidation` is hard because stale data can survive longer than the business expects.
- Many systems accept bounded staleness to gain performance and availability.

Eviction policies

- `LRU` evicts the least recently used item.

- `LFU` evicts the least frequently used item.
- `FIFO` evicts the oldest inserted item regardless of access pattern.
- The right policy depends on whether recency, frequency, or simplicity matters more.

Distributed caching risks

- `Distributed caches` improve sharing across instances, but they add network latency and failure modes.
- Hot keys can overload one shard or one cached item.
- Cache stampedes happen when many requests miss together and all hit the database at once.

Day-9 sample: LRU Cache

This repository includes a small LRU cache sample in both Python and Java.

Functional requirements

- Put key-value pairs into the cache
- Get values by key
- Move recently accessed items to the front of usage order
- Evict the least recently used item when capacity is exceeded

High-level components

- `Client`: reads and writes cached keys
- `Cache map`: stores key-to-node lookup
- `Usage order list`: keeps most recent and least recent ordering
- `Eviction logic`: removes the least recently used key when full

Data flow

1. A client reads or writes a key
2. The cache checks whether the key already exists
3. If found, the entry is moved to the most-recently-used position
4. If a new entry makes the cache exceed capacity, the least recently used entry is evicted
5. The client gets fast in-memory access for hot items

Project structure

```
Day-9/
  README.md
  lru-cache-sample/
    python/
      lru_cache.py
      demo.py
    java/
      LRUCache.java
```

```
Main.java
```

Run the sample

Python

```
cd lru-cache-sample\python
python demo.py
```

Java

```
cd lru-cache-sample\java
javac Main.java LRUCache.java
java Main
```

Interview takeaways

- Caches improve latency, but invalidation and consistency are the real design problems
- LRU is simple and effective when recent access predicts near-future reuse
- A high hit rate matters more than adding a cache in principle
- Distributed caches trade local speed for shared state and operational complexity
- Cache stampedes and hot keys must be handled explicitly in production systems

Next improvements

- Add TTL expiration support
- Add cache-aside loading behavior
- Add per-key locks to reduce stampedes
- Add metrics for hits, misses, and evictions
- Extend the sample into a Redis-backed caching layer

Source Code Appendix

Day-9/lru-cache-sample/python/lru_cache.py

```
from __future__ import annotations

from collections import OrderedDict
from dataclasses import dataclass, field
from typing import Generic, TypeVar

K = TypeVar("K")
V = TypeVar("V")


@dataclass
class LRUCache(Generic[K, V]):
    capacity: int
    _items: OrderedDict[K, V] = field(default_factory=OrderedDict, init=False)

    def __post_init__(self) -> None:
        if self.capacity <= 0:
            raise ValueError("capacity must be positive")

    def get(self, key: K) -> V | None:
        if key not in self._items:
            return None

        self._items.move_to_end(key)
        return self._items[key]

    def put(self, key: K, value: V) -> str | None:
        evicted: str | None = None
        if key in self._items:
            self._items.move_to_end(key)
        self._items[key] = value

        if len(self._items) > self.capacity:
            evicted_key, _ = self._items.popitem(last=False)
            evicted = str(evicted_key)
        return evicted

    def snapshot(self) -> list[tuple[K, V]]:
        return list(self._items.items())
```

Day-9/lru-cache-sample/python/demo.py

```
from lru_cache import LRUCache


def print_state(cache: LRUCache[str, str], label: str) -> None:
    print(label, cache.snapshot())


def main() -> None:
    cache = LRUCache[str, str](capacity=3)

    print("put A=alpha | evicted:", cache.put("A", "alpha"))
    print_state(cache, "  state:")

    print("put B=bravo | evicted:", cache.put("B", "bravo"))
    print_state(cache, "  state:")

    print("put C=charlie | evicted:", cache.put("C", "charlie"))
    print_state(cache, "  state:")

    print("get A ->", cache.get("A"))
    print_state(cache, "  state after touching A:")

    print("put D=delta | evicted:", cache.put("D", "delta"))
    print_state(cache, "  state:")

    print("get B ->", cache.get("B"))
    print("get C ->", cache.get("C"))
    print_state(cache, "  final state:")

if __name__ == "__main__":
```

```
    main()
```

Day-9/lru-cache-sample/java/LRUCache.java

```
import java.util.LinkedHashMap;
import java.util.Map;

public class LRUCache<K, V> {
    private final int capacity;
    private final LinkedHashMap<K, V> items;

    public LRUCache(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException("capacity must be positive");
        }
        this.capacity = capacity;
        this.items = new LinkedHashMap<>(16, 0.75f, true);
    }

    public V get(K key) {
        return items.get(key);
    }

    public K put(K key, V value) {
        items.put(key, value);
        if (items.size() > capacity) {
            K evictedKey = items.keySet().iterator().next();
            items.remove(evictedKey);
            return evictedKey;
        }
        return null;
    }

    public Map<K, V> snapshot() {
        return new LinkedHashMap<>(items);
    }
}
```

Day-9/lru-cache-sample/java/Main.java

```
public class Main {
    public static void main(String[] args) {
        LRUCache<String, String> cache = new LRUCache<>(3);

        System.out.println("put A=alpha | evicted: " + cache.put("A", "alpha"));
        printState(cache, "  state:");

        System.out.println("put B=bravo | evicted: " + cache.put("B", "bravo"));
        printState(cache, "  state:");

        System.out.println("put C=charlie | evicted: " + cache.put("C", "charlie"));
        printState(cache, "  state:");

        System.out.println("get A -> " + cache.get("A"));
        printState(cache, "  state after touching A:");

        System.out.println("put D=delta | evicted: " + cache.put("D", "delta"));
        printState(cache, "  state:");

        System.out.println("get B -> " + cache.get("B"));
        System.out.println("get C -> " + cache.get("C"));
        printState(cache, "  final state:");
    }

    private static void printState(LRUCache<String, String> cache, String label) {
        System.out.println(label + " " + cache.snapshot());
    }
}
```